The Mechanics of Constitutional Enforcement

Ratchets, Roles, Receipts, and Identity-Bearing State

Small topologies can carry large laws.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Governed systems frequently represent policy as runtime data evaluated by predicates: counters, flags, permission checks, validation functions, and conditional branches. This paper argues that the placement of governance semantics is itself a security and identity decision. A requirement represented as data depends on the completeness and integrity of every check that interprets it. The same requirement represented as state-transition topology constrains which executions are validly expressible. When that topology is canonically serialized and cryptographically bound into a constitution's identity, a canonical change produces a different identity except with negligible collision probability; custody verification can detect substitution against an expected identity, while a semantic court must still decide whether the change is a weakening.

The argument arises from a bounded repair in which a requirement for seven ordered, unique stages was initially represented by a counter. Seven repetitions of the same stage satisfied the quantity while violating the intended order and uniqueness. The repair added no new constructs: the same guards, effects, invariants, and kernel remained. The requirement was instead encoded as a state chain. The change did not enlarge the vocabulary; it relocated the semantics from data into topology.

From this observation, the paper develops three propositions. First, the **Semantic Placement Principle**: data-borne semantics are protected by the completeness of checks, while topology-borne semantics are protected by the closure of valid transitions. Second, the **Identity-Bearing Topology Principle**: when semantically relevant enforcement topology lies inside a declared canonical identity boundary, changing that topology changes the governed artifact's identity under the stated hash assumptions; a separate semantic judgment determines whether the change weakens the rule. Third, the **Ratchet Governance Conjecture**: a substantial class of governance requirements may be compilable into finite, monotone, role-gated, threshold-bounded transition systems.

Structural semantics do not eliminate the problem of a defective or malicious

executor. A nonconforming executor may abandon the automaton and impersonate its results. The paper therefore distinguishes structural adequacy, executor conformance, custody fidelity, and effect integrity. It argues that consequential effects should be mediated by independent authority brokers, external witnesses, transactional isolation, and absence-sensitive monitoring. The irreducible limit is not that malicious execution cannot be countered, but that no component can prove its own faithful behavior solely through evidence it alone controls.

The result is a theory of constitutional systems in which the kernel need not contain every governance concept. Its task may instead be to provide the smallest trusted mechanics into which governance can be compiled without losing identity, auditability, bounded refusal, or externally observable custody.

1. Introduction: Where Does the Rule Live?

A governed system may contain the right rule and still place it in the wrong part of the machine.

Consider a workflow requiring seven distinct stages to occur once each and in a prescribed order. One implementation records progress using a counter:

$$count \geq 7$$

The counter proves only that seven qualifying events were counted. It does not inherently preserve which events occurred, whether they were unique, or whether they followed the required order. If the same stage can be submitted seven times, then seven duplicates satisfy the representation while violating the policy.

The representation has preserved quantity and discarded identity.

The repaired implementation uses a state chain:

$$S_0 \to S_1 \to S_2 \to \cdots \to S_7$$

Each transition corresponds to one particular stage. A duplicate submission at (S_1) cannot advance the machine unless a transition from (S_1) back through the same stage exists. An out-of-order submission cannot advance unless the topology contains the corresponding edge. An eighth duplicate has no transition to take.

The illegal behavior is no longer merely prohibited by a predicate. It is absent from the transition relation.

This repair introduced no new kernel construct. The same four guards, four effects, four invariants, and kernel bytes remained. What changed was the location of the meaning. In the first version, the semantics lived in data interpreted by checks. In the second, they lived in the structure of possible execution.

That distinction is more than stylistic. It places the two designs under different failure regimes.

A checked rule can fail because the check is missing, skipped, misordered, bypassed, weakened, or applied to the wrong data. A structural rule can still fail if the executor is defective or malicious, but a conforming executor cannot traverse an edge that the admitted machine does not contain. The question changes from:

> Did every relevant operation remember to enforce the rule?

to:

> Does any valid path through the admitted system violate the rule?

A second consequence follows when the state topology is included in the canonical representation of the governing constitution. If the constitution's identity is derived from that representation, then adding a bypass edge, weakening a guard, or collapsing two stages changes the constitution's identity. The enforcement structure becomes part of the custody surface.

This yields the paper's central claim:

> Governance rules become stronger when they are relocated from checked propositions into identity-bearing topology.

This is not a claim that state machines are novel, nor that all governance can be reduced to linear workflows. It is a claim about the conjunction of semantic placement, structural reachability, cryptographic identity, and authority mediation.

The bounded observation motivates four questions:

1. Which governance properties can be encoded structurally using a small transition vocabulary?
2. What security changes when semantics move from runtime data into topology?
3. How does cryptographic custody alter the status of that topology?
4. What remains vulnerable when the executor itself is defective or hostile?

The answers suggest a design discipline for constitutional systems: enlarge the trusted kernel only when a required property is genuinely inexpressible, not merely inconvenient to compile.

2. Data-Borne and Topology-Borne Semantics

A semantic requirement can inhabit several layers of a system.

It may live in:

- a runtime value,
- a predicate over values,
- a transition relation,
- a type or capability boundary,
- a cryptographic identity,

- an external witness,
- or some composition of these.

These placements are not interchangeable. Each preserves different invariants and fails in different ways.

2.1 Data-borne semantics

A semantic requirement is **data-borne** when its meaning depends on interpreting mutable runtime values.

Examples include:

- a counter indicating completed approvals,
- a Boolean flag indicating verification,
- a list indicating which roles have acted,
- a timestamp indicating freshness,
- a string indicating a security state.

The value does not enforce its own intended interpretation. A counter stores cardinality. It does not know whether the counted items were unique. A Boolean stores one bit. It does not prove the event whose truth it purports to summarize.

Data-borne semantics therefore depend on a family of checks:

$$Permit(action) \iff P(data)$$

Their safety depends not merely on predicate (P), but on the proposition that every consequential path invokes the correct (P) over authentic data at the correct time.

The complete requirement is closer to:

$$\begin{aligned} &Safety = Correct(P) \\ &\wedge CompleteInvocation(P) \\ &\wedge Authentic(data) \\ &\wedge CorrectOrdering(P) \end{aligned}$$

A defect in any term may convert a valid local predicate into a globally ineffective policy.

2.2 Topology-borne semantics

A semantic requirement is **topology-borne** when it is encoded in the existence or absence of transitions.

Let a transition system be:

$$\mathcal{M} = (Q, \Sigma, \delta, q_0, F)$$

where Q is a set of states, Σ a set of admissible actions, and δ a transition relation.

A policy violation is structurally excluded when no valid transition path realizes it:

$$\nexists \pi \in Paths(\mathcal{M}) : Violates(\pi, P)$$

The system does not ask an executor to remember that stage four must follow stage three. It provides no admitted edge by which stage four follows stage one.

The semantics are carried by reachability.

This does not mean guards disappear. A transition may still be gated by evidence, role, threshold, identity, or resource conditions. The distinction is that the overall policy is not compressed into a summary value whose richer meaning must later be reconstructed.

2.3 The Semantic Placement Principle

The motivating case supports the following principle:

> **Semantic Placement Principle:** A requirement encoded as runtime data is protected by the completeness and integrity of its interpretation; the same requirement encoded as topology is protected by the closure of the admitted transition system.

This is not a universal ranking in which structure always dominates data. Some facts are intrinsically data-like. Amounts, identities, external measurements, signatures, and timestamps cannot be replaced wholesale by topology.

The principle instead asks whether the structural part of a policy has been unnecessarily flattened into data.

The relevant design question is:

> Which distinctions must the representation preserve for the policy to remain true?

A counter preserves quantity. A set preserves uniqueness. A sequence preserves identity and order. A transition graph preserves admissible succession. A role-partitioned graph preserves who may cause which succession. A cryptographic digest preserves a custody commitment to a particular representation.

A representation that discards a necessary distinction creates an attack surface aligned with what it forgets.

3. Semantic Homomorphism Attacks

The duplicate-stage failure can be understood as a broader class of representational attack.

Suppose a rich policy domain (X) is mapped into a simpler representation (Y):

$$f : X \to Y$$

The system then evaluates policy using only (f(x)). If two materially different states map to the same representation,

$$x_1 \neq x_2 \quad \text{but} \quad f(x_1) = f(x_2),$$

the enforcement layer cannot distinguish them without additional information.

In the seven-stage example:

$$f(sequence) = length(sequence)$$

Both a valid seven-stage sequence and seven duplicates have length seven:

$$f([A, B, C, D, E, F, G]) = 7$$
$$f([A, A, A, A, A, A, A]) = 7$$

The attack preserves what the representation measures while destroying what the policy means.

This paper calls such a failure a **semantic homomorphism attack**:

> A semantic homomorphism attack preserves the invariant visible to an enforcement representation while violating a richer invariant that the representation was intended to stand for.

Examples include:

- satisfying an approval count using repeated approval by one identity,
- satisfying a quorum count using aliases controlled by one authority,
- satisfying an "evidence present" flag with irrelevant evidence,
- satisfying a completion counter using repeated execution of one stage,
- satisfying a freshness field using an untrusted clock,
- satisfying a role list using duplicated or non-independent roles.

The attack is not necessarily an exploit against correct code. It may be a faithful exploitation of an impoverished representation.

Structuralization repairs the problem when it replaces the lossy mapping with a representation that retains the required distinctions.

3.1 Terminology boundary

"Semantic homomorphism attack" is used here as an architectural term for a many-to-one abstraction or quotient that preserves the enforcement-visible feature while collapsing a policy-material distinction. The paper does not claim that every mapping called f is a homomorphism under a specified algebraic structure. For formal publication, the author should either define the source and target structures and the preservation law, or retain the term while explicitly treating **representation collision** or **policy-distinction collapse** as the more general class.

4. Structuralization

Structuralization is the transformation of a semantic requirement from a value interpreted by checks into a constraint over admissible transitions, types, capabilities, or other reachable structures.

For ordered unique stages, structuralization transforms:

$$completed_count = n$$

into:

$$S_0 \xrightarrow{stage_1} S_1 \xrightarrow{stage_2} S_2 \cdots \xrightarrow{stage_n} S_n$$

For separation of duties, it may transform:

$$approver_role \neq requester_role$$

from a repeatedly evaluated predicate into separate capabilities held by distinct principals, such that no one principal possesses both transition authorities.

For evidence-before-effect, it may transform:

$$if evidence_valid : perform_effect()$$

into a system where the capability to perform the effect is reachable only after an evidence-admission transition.

Structuralization has several potential benefits.

First, it centralizes policy in the machine's shape rather than distributing it across call sites.

Second, it makes invalid succession visible through graph analysis.

Third, it enables mutation testing against transitions, guards, and terminal states.

Fourth, it may bring enforcement semantics inside a cryptographic identity boundary.

Fifth, it can reduce the semantic burden placed on ordinary runtime code.

Yet structuralization also has costs. It may increase state count, introduce product-state explosion, or obscure policies that are naturally quantitative. The relevant goal is not maximal structuralization. It is correct semantic placement.

5. Identity-Bearing Topology

Structural semantics become constitutionally stronger when the transition system is part of the governed artifact's identity.

Let a constitution be represented as:

$$IC = Canonicalize(Q, T, G, E, I, R)$$

where:

- (Q) is the state set,
- (T) is the transition relation,
- (G) is the guard set,
- (E) is the effect specification,
- (I) is the invariant set,
- (R) is the role and authority partition.

Let its identity be:

$$id(IC) = H(IC)$$

where (H) is a cryptographic hash over the canonical representation.

A semantic weakening produces a modified constitution (IC'). If the canonical representation changes, then under the declared canonicalization and collision-resistance assumptions:

$$IC' \neq IC \Rightarrow Pr[H(IC') = H(IC)] \leq \epsilon(\lambda)$$

Under those assumptions, a bypass edge, removed guard, or collapsed stage cannot preserve the expected constitutional identity except with negligible collision probability. This detects canonical difference; it does not classify the difference as weakening without semantic review.

The resulting property is not that mutation becomes impossible. Mutation remains physically possible. The property is:

> A semantic mutation whose material distinction is preserved by the declared canonicalization cannot remain custody-identical except within the stated collision bound.

This motivates the **Identity-Bearing Topology Principle**:

> When a policy-material transition distinction is preserved by the declared canonical representation and cryptographically bound into a constitution's identity, changing that distinction changes the expected identity under the stated hash assumptions; semantic review determines whether the change weakens the topology.

The difference between a runtime check and identity-bearing topology is therefore not merely where enforcement occurs. It is whether changing the enforcement semantics changes the governed object's cryptographic name.

A skipped runtime check may leave the constitution's identity untouched. A canonical change to a custody-covered transition graph changes the expected identity under the stated assumptions.

This creates a bridge between semantics and custody.

5.1 Structure is not automatically custody-covered

The principle depends on several conditions:

1. The representation must be canonical or unambiguous.
2. All semantically relevant topology must be included.
3. The hash must bind the exact admitted representation.
4. The runtime must load the constitution identified by that hash.
5. The verifier must compare against an independently trusted identity.
6. Semantic equivalences and serialization ambiguities must not permit hidden mutation.

If the runtime hashes one object but executes another, identity binding is ceremonial. If a parser interprets the same bytes differently across environments, the hash binds syntax without fixing meaning. If generated transitions are excluded from the digest, part of the operative topology remains outside custody.

Identity-bearing topology is therefore a systems property, not merely a call to a hash function.

5.2 Difference detection is not adequacy judgment

A changed digest detects a changed canonical representation relative to an expected identity; it does not reveal whether the change is a weakening, strengthening, repair, or irrelevant rewrite. That judgment requires the semantic court, a predecessor relation, an authorized amendment rule, and evidence that the canonicalization regime preserves the distinction being evaluated.

The identity claim is therefore conditional:

$$\begin{aligned} &Canonical(IC') \\ &\neq Canonical(IC) \\ &\implies Pr[H(IC') = H(IC)] \leq \epsilon(\lambda) \end{aligned} ,$$

under the declared canonicalization, domain separation, and collision-resistance assumptions. A hash is a commitment surface, not a semantic evaluator or a source of standing.

6. Four Courts and Four Distinct Claims

A constitutional system can be correct in one dimension and defective in another. At least four claims must be distinguished.

6.1 Semantic adequacy

Does the constitution encode a policy strong enough to support the intended governance claim?

A constitution that requires seven events but not seven unique ordered stages may be faithfully executed and perfectly custodied while remaining semantically inadequate.

6.2 Executor conformance

Does the implementation faithfully realize the admitted constitution?

A strong automaton does not help if the executor invents transitions, skips guards, or performs effects outside the transition mechanism.

6.3 Custody fidelity

Is the executed constitution the same constitution that was admitted, reviewed, and identified?

A conforming executor can faithfully run a substituted or weakened constitution.

6.4 Effect integrity

Can consequential external effects occur only through constitutionally authorized transitions?

An executor may calculate state correctly yet possess direct authority to perform unmediated effects.

A governed effect therefore requires a conjunction:

$$
\begin{aligned}
&GovernedEffect = Adequate(IC, P) \\
&\wedge Conforming(X, IC) \\
&\wedge Custodied(IC) \\
&\wedge Mediated(effect)
\end{aligned}
$$

These claims correspond to distinct courts.

The **semantic court** evaluates whether the constitution contains the required policy shape.

The **mechanical court** evaluates whether the executor implements the transition semantics correctly.

The **custody court** evaluates whether the admitted and executed artifacts are identical or validly related.

The **effect court** evaluates whether external consequences are possible only through authorized commitment.

No single court subsumes the others.

This separation prevents a dangerous form of laundering. A passing kernel verifier does not prove that a constitution is strong. A passing semantic profile does not prove that the executor obeys it. A matching hash does not prove that the hashed object is adequate. A valid internal receipt does not prove that no unreceipted external effect occurred.

6.5 Proof obligations by court

- **Court:** semantic
  Question: Does the admitted structure preserve the policy distinctions needed for the stated claim?
  Minimum evidence surface: requirement mapping, counterexamples, mutation fixtures, claim ceiling
  Does not establish: executor fidelity or effect mediation
- **Court:** mechanical
  Question: Does the executor implement the admitted transition semantics?
  Minimum evidence surface: conformance traces, negative transitions, replay or refinement evidence
  Does not establish: semantic adequacy of the constitution
- **Court:** custody
  Question: Is the executed object the admitted object or a validly receipted successor?
  Minimum evidence surface: canonical identity, exact artifact, lineage and amendment receipts
  Does not establish: that the object is adequate or safely executed

- **Court:** effect
  Question: Can the claimed consequential effect occur only through authorized commitment?
  Minimum evidence surface: capability boundary, externally observed outcomes, classified partial effects
  Does not establish: correctness of hidden or unobserved behavior

These are court roles, not necessarily four software services. One implementation may evaluate several roles only if its authority, evidence, and failure-domain assumptions remain explicit. Independence is not achieved merely by using another process owned by the same actor with the same keys and observation boundary.

7. Governance as Ratchet Automata

The bounded case suggests that many governance requirements share a common shape.

They are frequently:

- monotone,
- ordered,
- role-gated,
- evidence-gated,
- threshold-bounded,
- consumption-bounded,
- terminal or semi-terminal,
- reversible only through explicit authority,
- and sensitive to replay or duplication.

These properties resemble a ratchet: advancement occurs through discrete clicks; ordinary execution cannot silently move backward; reversal, when permitted, requires a separate governed mechanism.

A **ratchet automaton** may be represented as:

$$\mathcal{G} = (Q, R, A, E, \Theta, \delta, q_0, F)$$

where:

- Q is a finite set of governance states,
- R is a set of roles or authority classes,
- A is a set of proposed actions,
- E is a set of admissible evidence classes,
- Θ is a set of thresholds or bounded resources,
- δ is a role-, evidence-, and threshold-gated transition relation,
- q_0 is the initial state,
- F is a set of terminal, committed, refused, revoked, or recovery states.

A transition may take the form:

$$\delta(q, a, r, e, \theta) = q'$$

only when:

- role (r) has authority for action (a),
- evidence (e) is admissible,
- threshold condition θ is satisfied,
- the transition exists from state (q),
- replay and uniqueness constraints permit advancement.

7.1 Structural mappings

Several common governance requirements map naturally into this form.

Ordered approval

$$Draft \rightarrow Reviewed \rightarrow Approved \rightarrow Committed$$

Unique stage completion

Each stage has one outgoing advancement edge and no self-repeating advancement edge.

Separation of duties

Different transitions require capabilities held by distinct role partitions.

Evidence before effect

The effect-bearing state is unreachable until evidence admission has occurred.

Bounded retry

Each failure consumes a finite transition budget.

Quorum

Advancement requires a threshold over distinct admitted identities.

Monotone activation

Activation is a terminal or forward-only state unless an explicit revocation path exists.

Recovery

Incomplete states lead into a recovery subgraph rather than being interpreted as success.

Refusal

Invalid or unsupported requests terminate in an explicit refusal state that performs no consequential effect.

7.2 The Ratchet Governance Conjecture

The broader conjecture is:

> A substantial class of practical governance requirements can be compiled into finite state chains or graphs combined with role partitions, evidence gates, threshold conditions, and bounded resources.

This is not yet a theorem.

The motivating evidence is bounded: one specification family, eleven constitutions, seventy-seven operations, and linearizable workflows. The demonstrated sufficiency of a small vocabulary establishes a case, not a universal expressive result.

The research task is to characterize the class precisely.

A possible target property class is:

$$FiniteState \times RolePartition \times ThresholdGuards \times MonotoneEffects$$

The useful result may be either positive or negative.

A positive characterization would show which governance policies can be compiled without kernel growth.

A negative boundary would identify the first property that cannot be represented without an additional primitive, an external oracle, or unacceptable state expansion.

Both outcomes would guide constitutional kernel design.

8. The Nonconforming Executor Problem

Topology makes violations unrepresentable to a conforming executor. It does not physically compel arbitrary machine code to remain conforming.

Suppose the admitted automaton contains:

$$S_0 \to S_1 \to S_2$$

and no edge from (S_0) directly to (S_2).

A faithful executor cannot validly take the absent transition. A defective or malicious executor may nevertheless execute code equivalent to:

$$state := S_2$$

and perform the effect associated with (S_2).

At that moment, it has not traversed an illegal edge within the automaton. It has abandoned the automaton and impersonated its result.

This distinction should be stated precisely:

> Topology constrains execution inside the admitted model. It does not, by itself, imprison the physical machine hosting the model.

The observation does not defeat structural semantics. It identifies their trust boundary.

A type system cannot stop an attacker who replaces the compiler. A proof cannot control hardware that ignores the verified program. A protocol cannot restrain a participant that possesses direct external authority beyond the protocol's mediation.

The practical question is therefore not whether nonconforming execution is metaphysically impossible. It is whether a nonconforming executor can:

1. cause consequential effects,
2. exceed a bounded damage radius,
3. impersonate authorized execution,
4. erase evidence of divergence,
5. prevent external detection,
6. or rewrite constitutional history.

A well-designed system should make each progressively harder.

9. The Irreducible Limit

There is a narrow irreducible limit.

If one actor controls:

- the executor,
- the effect-producing capability,
- the state store,
- the signing keys,
- the clock,
- the logs,
- the verifier,
- the witness,
- and the alert channel,

then it may be able to perform an unauthorized effect and fabricate every locally available account of what occurred.

No component can establish its own faithful execution solely through evidence that it alone creates and controls.

This may be stated as an epistemic limit:

> A self-reporting executor cannot, by self-report alone, distinguish faithful execution from a sufficiently capable fabrication of faithful execution.

The limit does not imply that trustworthy execution is impossible. It implies that trust cannot be eliminated by recursively asking the same compromised boundary to certify itself.

The response is architectural separation:

- distribute authority,
- externalize observation,
- narrow capabilities,
- diversify witnesses,
- preserve independent custody,
- and make silence itself evidentiary.

Trust is not removed. It is divided, bounded, and made contestable.

10. Constitutional Mediation

The strongest practical defense is to prevent the executor from possessing unilateral authority over consequential effects.

Instead of:

$$Constitution \rightarrow Executor \rightarrow Effect$$

the system should use a mediated architecture:

$$\begin{aligned} &Constitution \\ &\rightarrow Court \\ &\rightarrow AuthorityBroker \\ &\rightarrow BoundedExecutor \\ &\rightarrow StagedEffect \end{aligned}$$

The executor proposes an operation. It does not authorize itself.

A proposal may contain:

$$\begin{aligned} Proposal = (&constitution_hash, current_state, transition, evidence, \\ &requested_effect, input_hash, nonce) \end{aligned}$$

An independent broker verifies:

- the constitutional identity,
- the present state,
- the existence of the transition,
- the relevant role authority,

- evidence admissibility,
- replay status,
- resource bounds,
- and the proposed effect's scope.

Only then does the broker release a narrowly scoped capability.

The executor may still calculate incorrectly. It may still crash, lie, stall, or submit malformed evidence. But it should not be able to transform that failure directly into an admitted effect.

The design principle is:

> A defective executor may propose falsehood, but it should lack unilateral authority to make the falsehood consequential.

10.1 Capability release

The broker can release authority scoped to:

- one operation,
- one artifact hash,
- one destination,
- one workspace,
- one transaction,
- one time window,
- one maximum resource budget,
- one expected output class.

Authority should expire after use or after a bounded deadline.

The executor is treated as an untrusted worker, not as the sovereign interpreter of the constitution.

10.2 Reserve, execute, verify, commit

Consequential operations can be decomposed:

$$\begin{aligned} &Propose \\ &\rightarrow Reserve \\ &\rightarrow ExecuteInIsolation \\ &\rightarrow Verify \\ &\rightarrow Commit \end{aligned}$$

The reserve step establishes an externally visible intention without yet producing the final effect.

Execution occurs in a sandbox, copy-on-write environment, transactional namespace, or staged publication area.

Verification evaluates the resulting state and evidence.

Commit is performed or released by an authority distinct from the worker that produced the candidate result.

If execution fails, stalls, or contradicts its reservation, the system enters recovery or refusal rather than silently interpreting partial work as success.

10.3 Effects as capabilities

The most consequential effects should not be ordinary function calls available to the executor.

They should be capabilities whose possession is itself governed.

Examples include:

- committing a filesystem transaction,
- publishing an artifact,
- activating installed software,
- releasing a signing key,
- changing an access-control state,
- transmitting protected information,
- accepting a financial transfer,
- updating an authoritative manifest.

This moves enforcement from "remember to check before calling" toward "the call cannot be made until authority is released."

Structure closes the forbidden road. Capability mediation removes the executor's independent machinery for building another.

10.4 Effect atomicity and classified remainders

`Reserve -> ExecuteInIsolation -> Verify -`

`> Commit` is a target protocol, not proof that the real-world effect is atomic. Some effects are irreversible, externally visible during staging, or only compensable after partial completion. A truthful implementation must distinguish at least:

- **NO_EFFECT:** no consequential boundary was crossed;
- **STAGED:** candidate state exists but is not authoritative;
- **COMMITTED:** the authorized effect was durably admitted;
- **ABORTED:** the reservation ended without the intended effect;
- **PARTIAL_EFFECT:** some consequence occurred and exact rollback is not established;
- **COMMIT_AMBIGUOUS:** the observer cannot establish whether commit occurred;
- **COMPENSATED:** a later action counteracted an effect without pretending it never happened.

Validation should precede mutation where possible, corrupt state must not be treated as missing state, read-only verification must not mutate the subject, and multi-artifact commits need either actual atomicity or truthful remainder classification. This is the paper's direct adjacency to the Integrity Enforcement Layer; adjacency does not prove that a particular runtime implements IEL.

11. Independent Evidence and Witnessing

An executor's own receipt is useful but insufficient as conclusive evidence of executor faithfulness.

A stronger design requires evidence that the executor cannot forge alone.

Possible evidence includes:

- independent transition recomputation,
- deterministic replay,
- pre-state and post-state hashes,
- signed broker authorizations,
- hash-chained receipts,
- witness signatures,
- externally observed effects,
- challenge-response transcripts,
- hardware-backed measurements,
- quorum attestations,
- public or organizational transparency logs.

The guiding rule is:

> The actor whose conduct is being evaluated should not be the sole author of the evidence that vindicates that conduct.

A transition receipt might bind:

$$Receipt = H(IC, q, a, e, q', effect, previous_receipt, nonce)$$

The witness can verify that:

- the constitution matches the admitted identity,
- state (q) was current,
- action (a) was admissible,
- evidence (e) satisfied the transition,
- state (q') follows from the constitution,
- the external effect corresponds to the committed result,
- the receipt extends the expected chain.

A fabricated post-state may still be proposed, but it cannot obtain the independent commit signature unless the broker or witness is also compromised.

11.1 External effect observation

Internal state evidence is not enough when the real claim concerns the world outside the executor.

If the executor claims that a file was not published, an independent observer may inspect the publication surface.

If it claims that a package was activated only after approval, the activation substrate should require a broker-issued capability.

If it claims that a protected record was not transmitted, network mediation must occur outside the executor's direct control.

Effect integrity requires observing or controlling the effect boundary, not merely trusting a log about it.

11.2 Independent commitments and receipt claim ceilings

The illustrative receipt hash is useful as a bundle commitment, but one digest should not collapse distinct governance decisions. Applying the separation discipline of *The Three Hash Rule*, a realization should distinguish:

- H_s: state identity and transition outcome;
- H_c: admissibility, role, evidence, replay, and authority judgment;
- H_m: semantic regime and interpretation of the constitution/effect;
- optional H_j: phase or jurisdictional locality;
- H_r: receipt binding across the preceding commitments, predecessor receipt, nonce, and context.

The receipt must also state what was observed and by whom. A signed internal assertion proves at most that a key signed the assertion. An external-effect claim requires an observer or mediator whose custody and failure assumptions match that claim.

In *Reasoning by Way of Receipts* terms, the receipt should bind premises, decision procedure, evidence, limits, outcome, and unresolved remainder. It should never be promoted from "this transition record verifies" to "no alternate unreceipted effect occurred" without an effect-boundary proof.

12. Absence-Sensitive Monitoring

A compromised executor can suppress its own alerts. Alerting must therefore live outside the suspected boundary.

External monitors can expect:

- a completion receipt after a reservation,
- a signed heartbeat before a deadline,
- a monotonically increasing sequence number,
- publication of each committed transition,

- a matching abort receipt after failed execution,
- a corresponding constitutional commit for each observed effect.

This enables **absence alerts**.

The monitor need not wait for the executor to confess. It can detect that an expected event failed to appear.

Examples include:

> A transition was reserved, but no valid completion or abort receipt arrived.
>
> A capability was released, but no terminal receipt was produced before expiry.
>
> An external effect exists without a corresponding commit record.
>
> The receipt chain advanced in one mirror but not another.
>
> The active artifact hash differs from the admitted manifest.
>
> A worker stopped responding while still holding a live reservation.

Absence-sensitive monitoring is particularly valuable because fabricated positive evidence is often easier than fabricating consistent silence across independent observers.

An alert may trigger:

- capability revocation,
- transaction rollback,
- workspace quarantine,
- key rotation,
- process termination,
- evidence preservation,
- operator review,
- or system-wide refusal.

A system cannot always prevent the first deviation. It can make the deviation bounded, visible, and unable to mature into accepted history.

12.1 Conditions for treating absence as evidence

Silence becomes evidence only relative to a declared observation contract. The monitor must know which event was required, which observer and channel should carry it, the deadline and clock source, whether delivery is reliable, how partitions and observer failure are classified, and what conclusion non-arrival permits. Without these conditions, absence may indicate omission, delay, partition, crash, censorship, or monitor failure rather than executor misconduct.

An absence alert is therefore evidence of a violated expectation under a bounded observation model. It is not direct proof of the hidden cause.

13. Formal Verification and Attestation

Formal verification may establish that an executor implementation refines an automaton specification. This can substantially reduce ordinary defects and narrow the trusted implementation.

Yet verification does not abolish assumptions.

The proof may depend on:

- the proof checker,
- the formal model,
- the compiler,
- the build process,
- the runtime,
- the operating system,
- the hardware,
- and the fidelity of external inputs.

Reproducible builds can connect verified source to a deterministic binary. Measured boot can connect the binary to a running environment. Hardware-backed keys can bind receipts to a measured execution state.

Each step reduces ambiguity, but none should be treated as an oracle of semantic truth.

A measured binary can faithfully contain a defect. A verified model can omit an environmental behavior. A hardware root can fail or be subverted. A remote attestation can prove that a particular binary was loaded without proving that the wider system behaved correctly.

Attestation is therefore one court among several.

The goal is not an infinite regress of perfect proof. It is a small, explicit, diverse, and reviewable trust base whose failure modes are bounded and externally detectable.

14. Mutation Courts

A minimal kernel can validate weak constitutions. Mechanical expressibility does not establish semantic adequacy.

A second layer must therefore test whether the required policy shape is actually present.

A **mutation court** evaluates whether materially weakened constitutions are rejected or differently identified.

Mutation classes may include:

- duplicated stages,
- reordered stages,

- skipped evidence,
- forged role assignment,
- threshold reduction,
- repeated resource consumption,
- rollback after terminal commitment,
- removal of a refusal state,
- addition of a bypass edge,
- weakening of a transition guard,
- substitution of an untrusted external assertion,
- rehashing of a maliciously altered constitution.

For each mutation, the court should ask:

1. Is the mutation structurally executable?
2. Does it change the constitutional identity?
3. Which court detects it?
4. Can the executor produce an effect before detection?
5. Does detection depend on a check controlled by the executor?
6. Is the resulting failure bounded and recoverable?
7. Can the mutation be accepted as a different, explicitly weaker constitution?

The last question matters. A constitutional system need not prohibit weak constitutions from existing. It must prevent them from inheriting stronger standing than they earned.

Mutation courts therefore police claim boundaries as much as execution boundaries.

15. Expressive Boundaries

The ratchet model has known limits.

15.1 Cross-entity truth

A constitution can bind another artifact by hash. It can require evidence that a foreign artifact possesses a particular identity. It cannot infer the foreign artifact's truth merely from the binding.

Federation by digest is not a semantic join.

Cross-entity claims require additional admission procedures, shared courts, trusted witnesses, or independently verifiable proofs.

15.2 Cryptographic verification

An automaton can contain a transition labeled "signature verified." That representation does not perform cryptographic verification.

The verification must occur in an admitted cryptographic component, and the transition must bind its result.

This is the difference between representing an assertion and earning it.

15.3 Time and freshness

Time does not arise from finite topology alone.

A policy such as "complete within ten minutes" requires a clock or temporal witness. Freshness may require comparison against local, organizational, public, peer-to-peer, or independently mirrored manifests.

The topology can govern how temporal evidence is admitted and what follows from expiration. It cannot manufacture trustworthy time.

15.4 Arbitrary partial orders

A linear chain represents total order economically. A complex partial order may require a product state for every combination of completed dependencies.

The result can be exponential state growth.

Other formalisms–Petri nets, event structures, temporal logics, dependency graphs, or compositional automata–may be more appropriate.

The relevant question is whether these should become kernel primitives or compile into a retained lower-level form.

15.5 Liveness

A topology can exclude invalid paths without guaranteeing that a valid path will complete.

A system may remain forever in an intermediate state.

Deadlines, fairness assumptions, recovery transitions, watchdogs, and bounded refusal are required to address liveness.

15.6 Side channels and unmediated effects

A constitution cannot govern effects that occur through channels it does not mediate or observe.

If an executor possesses unrestricted network, filesystem, device, or memory authority, it may act outside the constitutional path.

The authority boundary must therefore match the claim boundary.

15.7 State explosion

A rule may be structurally expressible yet operationally impractical.

This creates an intermediate category between fundamental inexpressibility and mere inconvenience:

expressible only at unacceptable representational cost.

Kernel growth may sometimes be justified as a compact primitive for a recurring structure. Such growth should be accompanied by a formal account of what complexity it removes and what new trust surface it creates.

16. The Kernel-Growth Razor

Every primitive added to a constitutional kernel enlarges the surface that must be frozen, implemented, tested, verified, custodied, and trusted.

A new construct should therefore face a structured burden of proof.

Before adding it, ask:

1. Is the required property genuinely inexpressible using the existing state, role, threshold, evidence, and effect vocabulary?
2. Is it expressible only through unacceptable state-space expansion?
3. Does it depend on an irreducible external oracle?
4. Is the proposed construct semantic, or is it syntactic convenience?
5. Could a compiler or profile layer produce the same result?
6. What new executor behavior must enter the trusted base?
7. What mutation court accompanies the construct?
8. What weaker constitutions become expressible because of it?
9. Can its authority be separated from its representation?
10. Does it preserve deterministic identity and custody?

This yields three broad categories.

Physics: the property cannot be represented or mediated without extending the model.

Complexity management: the property is representable but requires a more compact or compositional form.

Debt wearing a feature's clothes: the property is already expressible, but tooling has not made the expression convenient.

The razor is skeptical without being hostile to growth. Minimality is not an aesthetic absolute. It is an epistemic advantage: every omitted primitive is one less semantic mechanism whose behavior must be trusted.

17. Implications for Constitutional Systems

The argument has direct consequences for systems such as GOORRT, CGSOS, Jnana, constitutional storage layers, governed inference systems, and software-admission frameworks.

17.1 GOORRT

A realization system should distinguish between:

- whether a source constitution is structurally representable,
- whether the realization preserves its semantics,
- whether the resulting artifact is identity-bound,
- and whether the runtime mediates consequential effects.

A successful realization is not merely a parser accepting a source grammar. It is a custody-preserving transformation whose semantic commitments survive into reachable execution.

17.2 CGSOS

Any capability added beyond a minimal constitutional vocabulary should be tested against the kernel-growth razor.

The central question is not:

> Would this primitive make governance easier to write?

It is:

> What policy becomes possible that could not previously be represented, compiled, or externally mediated within acceptable bounds?

Each added primitive should bring:

- a precise semantic definition,
- a custody representation,
- an executor-conformance court,
- a mutation court,
- an authority model,
- and a claim ceiling.

17.3 Jnana and governed execution substrates

A governed execution substrate should treat application code as a bounded worker rather than as the authority that defines its own compliance.

The substrate should mediate:

- installation,
- activation,
- execution,
- protected storage,
- network access,
- signing,
- publication,
- recovery,
- and rollback.

The executor may fail, but it should not be able to silently convert failure into standing.

The desired system property is:

> A nonconforming executor may compute incorrectly, but it cannot independently produce an admitted consequential effect; incomplete, contradictory, or unreceipted behavior becomes externally detectable and recoverable.

17.4 Constitutional storage

Identity-bearing topology aligns naturally with content-addressed storage.

The full hash can serve as the artifact's hard storage identity, while semantic names remain in an index layer. A constitutional topology then has a stable, integrity-checkable true name.

The semantic index can say what the constitution is for. The hash says which exact constitution it is.

17.5 Governed AI agents

An AI agent should not directly possess all capabilities associated with its proposed actions.

The model may suggest a transition. An independent policy and authority layer should determine whether the transition is admitted, whether evidence is sufficient, and whether the requested capability should be released.

The agent is an engine of proposal, not the final court of authority.

This architecture is especially important because semantic fluency can create the appearance of justified action without independently establishing custody, authorization, or effect integrity.

17.6 Governed Meaning Base and historical system names

The **Governed Meaning Base (GMB)** is the current term in the author-confirmed lineage `CCF -> CMB -> GMB`. In this architecture, GMB can retain the requirement, structural representation, counterexamples, mutation fixtures, court judgments, implementation claims, receipts, and claim movement without making any of them authoritative merely by storing them.

GOORRT, CGSOS, and Jnana are retained here as source-native system or architecture candidates with unresolved implementation and identity standing. Jnana is not silently merged into GMB, and the named systems are not treated as deployed proof of the paper's propositions. Current synthesis should say GMB when it means the governed semantic substrate; exact historical names remain unchanged in source-scoped discussion.

18. Research Program

The bounded observation can be developed into a broader research program.

18.1 Expressibility characterization

Construct a corpus of governance requirements and classify each as:

- directly structurally expressible,
- expressible through composition,
- expressible with state explosion,
- dependent on an external oracle,
- or genuinely inexpressible in the candidate vocabulary.

18.2 Comparative representations

Encode equivalent policies as:

- runtime counters and flags,
- predicate systems,
- state chains,
- role-partitioned automata,
- capability graphs,
- richer concurrent models.

Compare:

- mutation resistance,
- verifier complexity,
- audit surface,
- state-space cost,
- custody coverage,
- and executor trust requirements.

18.3 Minimal-basis inquiry

Determine whether a small set of guards, effects, invariants, and topology operators forms a useful basis for governance compilation.

The goal need not be mathematical universality. A practically sufficient basis for a well-characterized governance class would already be valuable.

18.4 Compiler correctness

Develop a compiler from higher-level governance descriptions into ratchet automata.

The compiler should emit:

- the constitution,
- the constitutional identity,

- a semantic correspondence proof or witness,
- mutation fixtures,
- executor test vectors,
- and claim-bounded receipts.

18.5 Mediated effect courts

Test whether an untrusted executor can cause any externally visible effect without an independently authorized commit.

The strongest negative tests should include:

- forged completion,
- dropped commit acknowledgment,
- substituted abort,
- stale evidence,
- replayed authorization,
- forged session identity,
- unreceipted external effect,
- deliberately stalled execution,
- and compromised local logging.

18.6 Plural witnessing

Study the tradeoff between one trusted broker and multiple independent witnesses.

Plurality may reduce single-boundary compromise but introduces coordination, availability, and disagreement problems.

The goal is not maximal decentralization. It is enough independence that no actor can both commit the effect and author the sole accepted history of that effect.

19. Discussion: Toward a Semantic Physics of Governance

The phrase "semantic physics" should be used carefully.

The present result is not a universal law of meaning. It is a search for stable relations among:

- representation,
- reachability,
- identity,
- authority,
- mutation,
- and observable effect.

The analogy is useful when it disciplines rather than inflates the claim.

A physical law does not depend on a programmer remembering to invoke it. The aspiration of structural governance is similar: move recurring semantic obligations out of scattered discretionary checks and into the invariant mechanics of the system.

Yet constitutional computing cannot eliminate interpretation. A semantic court must still decide which structure is adequate. External evidence must still be admitted. Executors must still be bounded. Effects must still be mediated. Roots of trust must still be chosen.

The emerging picture is therefore layered.

The kernel provides mechanics: what kinds of transitions can be enforced.

The constitution provides topology: which transitions exist for this governed system.

The semantic court provides statics: which structures must be present for a claim to stand.

The custody layer provides identity: which exact structure was admitted.

The executor provides realization: how the structure is enacted.

The broker provides authority: which effects may become consequential.

The witness provides observability: whether execution and effect correspond to the admitted history.

The alert layer provides temporal accountability: whether expected evidence appeared when required.

No layer is the whole system. Their composition is the constitutional regime.

Semantic Placement and Mediated Enforcement Mechanics

Across the completed P1 corpus, The Mechanics of Constitutional Enforcement has a non-duplicative role: it is the **mechanical theory of where constitutional meaning is placed and how that meaning is carried through identity-bearing structure, adjudication, mediation, effects, and receipts**. It does not supply the whole constitution, legitimacy theory, development process, or universal governance implementation.

- **Adjacent work:** *The Integrity Enforcement Layer*
  Supplies: A general effect-mediation and protected-refusal architecture.
  Enforcement-mechanics boundary: This paper analyzes placement, ratchet topology, court separation, and evidence obligations within such mediation.
- **Adjacent work:** *Governance as Compilation*
  Supplies: Staged transformation, lowering, and semantic remainder.

Enforcement-mechanics boundary: Compilation constructs enforceable structures; enforcement still depends on mediation and observed effects.

- **Adjacent work:** *From CI/CD to ACG*
  Supplies: A constitutional development and assurance process.
  Enforcement-mechanics boundary: ACG organizes the lifecycle; this paper explains the enforcement mechanisms evaluated within it.
- **Adjacent work:** *Legitimacy Without Self-Ratification*
  Supplies: Antecedent authority, independence, and standing ceilings for protected transitions.
  Enforcement-mechanics boundary: Enforcement can be mechanically complete while enforcing an illegitimate rule or self-ratifying verdict.
- **Adjacent work:** *Universal Governance Kernel (UGK)*
  Supplies: A candidate reusable governance-kernel realization.
  Enforcement-mechanics boundary: UGK may instantiate the mechanics; no single kernel is identified with the theory.

A portable enforcement case is:

$$M = (rule, placement, subject, prestate, identity, mediator, \\ courts, executor, effect_path, witnesses, receipts, \\ remainder, poststate) \quad .$$

The rule may be data-borne, topology-borne, or mixed. Moving a rule into topology can reduce some substitution attacks while introducing opacity, rigidity, state explosion, compiler dependence, and bypass risk. Structural difference detection only shows that structures differ; semantic adequacy still requires a declared model and evidence.

The four court claims remain separate: semantic adequacy of the encoded rule, executor conformance, custody fidelity, and effect integrity. A pass in one court cannot stand in for another. Mediation further separates reserve, execute, observe, verify, and commit. Unknown or partially committed effects produce classified remainder rather than a synthetic success or failure.

The universal GSM can type `RULE_PLACED_IN_DATA`, `RULE_PLACED_IN_TOPOLOGY`, `STRUCTURALIZED_AS`, `IDENTITY_BEARING_STATE`, `MEDIATED_BY`, `ADJUDICATED_FOR_ADEQUACY`, `ADJUDICATED_FOR_CONFORMANCE`, `ADJUDICATED_FOR_CUSTODY`, `ADJUDICATED_FOR_EFFECT`, `BYPASSED_BY`, `LEAVES_EFFECT_REMAINDER`, and `COMMITTED_AS`. These relations keep encoded law, enforcement, and observed outcome distinct.

This integration establishes four editorial commitments: adopt semantic placement and mediated enforcement mechanics as the official corpus role; adopt the enforcement-case envelope; preserve the four court claims as independent

assurance dimensions; and require classified remainder for uncertain or partial effects.

Enforcement Freshness and Coverage Closure

The fixed-seed comparison with GCML, Semantic Legitimacy Before Context, the Identity Spine Doctrine, and Governance Coverage Gaps reveals two additional prerequisites for an enforcement claim: **the enforced meaning must remain current, and all relevant effect paths must actually cross the mechanism**.

$$E = (rule_identity, standing_version, procedural_carrier, \\ placement, mediators, effect_paths, coverage, \\ observations, invalidation, remainder) \quad .$$

GCML can carry the admitted rule and its obligation status into runtime structures. SLBC supplies purpose-specific standing. Identity Spine decisions can split, merge, supersede, or contest the rule's semantic subject. Enforcement must bind those exact versions and refuse or quarantine stale structures when identity or standing changes.

Governance Coverage Gaps tests capability acquisition and effect routes outside the declared mediator. A mechanically correct court and ratchet can coexist with an administrative credential, retry path, plugin, build hook, emergency interface, or provider operation that bypasses it. Coverage is therefore an independent, interval-bounded court over the realized system.

The architecture should distinguish
`RULE_CURRENT_BUT_UNCOVERED`,
`RULE_STALE_BUT_MEDIATED`, `RULE_CURRENT_AND_COVERED`, `EFFECT_PATH_UNKNOWN`,
and
`RULE_INVALIDATED_PENDING_REBUILD`. None proves legitimacy or correctness, but each locates a different repair.

The universal GSM can type
`ENFORCES_RULE_AT_STANDING_VERSION`,
`INVALIDATED_BY_IDENTITY_CHANGE`, `QUARANTINES_STALE_TOPOLOGY`,
`DISCOVERS_BYPASS_EFFECT_PATH`, `COVERED_DURING_INTERVAL`, `REQUIRES_ENFORCEMENT_REBUILD`,
and
`LEAVES_EFFECT_PATH_UNKNOWN`.

This randomized review establishes four editorial commitments: add exact semantic and standing versions to enforcement cases; require invalidation after identity or admission changes; make capability-surface coverage an independent assurance dimension; and adopt the five diagnostic states for freshness and coverage.

20. Universal GSM Integration

The paper's core entities should remain distinct in the universal governed semantic model: requirement, policy distinction, source representation, target representation, abstraction map, collision witness, structuralization transform, constitution, constitution revision, state, transition, guard, role, evidence class, effect specification, executor, broker, capability, reservation, staged result, effect, witness, monitor, receipt, court judgment, mutation, canonicalization regime, identity commitment, trust assumption, and claim standing.

High-value typed relations include:

- a requirement **PRESERVES_DISTINCTION** a named policy distinction;
- a representation **COLLAPSES_DISTINCTION** when two materially different cases share an enforcement-visible image;
- a structuralization **TRANSFORMS** one representation into another under a declared correspondence claim;
- a constitution revision **HAS_CANONICAL_IDENTITY** under a named regime;
- an executor **REALIZES_CANDIDATE** a constitution, without identity implication;
- a broker **MEDIATES** a capability release;
- a receipt **BINDS** a transition judgment and an observed or claimed effect;
- a court judgment **ESTABLISHES_WITHIN_SCOPE** one claim and explicitly excludes the other court claims;
- a mutation fixture **COUNTERTESTS** a policy or identity claim;
- an implementation result **MOVES_CLAIM** from proposed to supported, weakened, refused, or unresolved.

Lexical similarity, shared hashes in different domains, or successful execution does not establish these relations. Consequential edges require exact source or receipt support.

21. Corpus Placement

This paper supplies the mechanical bridge among several corpus layers:

1. *Admissibility Engineering* asks whether incorrect realizations can be excluded structurally rather than discouraged in prose.
2. *The Integrity Enforcement Layer* makes validation-before-mutation, corrupt-not-missing, scoped verification, read-only inspection, and truthful effect trails part of the enforcement spine.
3. *The Three Hash Rule* prevents state identity, admissibility, meaning, locality, and receipt binding from collapsing into one digest.
4. *Reasoning by Way of Receipts* treats receipts as claim-bounded reasoning artifacts rather than generic logs.

5. *A Formal Theory of Work Custody* separates constitution, version family, executable realization, receipt, and effect identities.
6. *Constitutional Storage* provides content-addressed custody and lineage surfaces without turning a stored hash into semantic adequacy.
7. GMB retains the semantic and evidentiary fabric; it is not the effect authority.

The current UGK/IEL skill informed these proof boundaries as a compressed navigation aid. Its r165 pedagogical lane is an update candidate, not canonical law or publication evidence, and its verification results are not evidence for this paper's source-reported bounded repair.

22. Realization Funnel

Stage	Enforcement-mechanics role
Possibility	Enumerate data, predicates, sequences, graphs, types, capabilities, role partitions, witnesses, effect protocols, failure cases, and irreducible external oracles.
Specification	Declare preserved distinctions, court claims, canonicalization, transition semantics, evidence classes, authority, receipt schema, effect boundary, liveness requirements, and falsifiers.
CM1	Retain all admissible representations, including weak, state-explosive, unavailable, non-atomic, costly, or easily captured configurations.
CM2	Prefer configurations that preserve required distinctions with minimal trusted vocabulary, bounded authority, recoverable lineage, diverse evidence, and explicit remainders.
Instance Configuration	Select the exact constitution, compiler/profile, executor, broker, witnesses, keys, clocks, effect adapters, monitors, budgets, and court procedures.
Implementation Realization	Compile or author the topology, instantiate components, mediate capabilities, execute protocols, and emit receipts.

Stage	Enforcement-mechanics role
Evidence	Preserve fixtures, mutations, traces, canonical identities, commitment openings, effect observations, absence contracts, failures, and court judgments.
Revision	Repair representation or implementation, authorize successor constitutions, migrate state, retain predecessor identity, and record claim movement.

The source-reported repair reaches Implementation Realization rhetorically, but the current paper package does not bind the external fixtures needed to establish that stage independently.

Enforcement as Measured Coverage of Consequential Paths

Twelve downstream uses apply the paper to agent interfaces, admissibility engineering, infrastructure, cyber-physical syscalls, ACG, GCML, operating systems, the Integrity Enforcement Layer, boundary accountability, APD containment, UGK, and structural warrant. These uses confirm the paper's core contribution: rules must be placed in structures that mediate consequential paths. They also show that topology, ratchets, courts, receipts, and capabilities do not establish complete mediation merely by being present.

The enforcement case is:

$$\begin{aligned} E^* = (&rule_identity, subject, operation, realization_paths,\\ &entry_points, monitor, gates, capabilities, roles,\\ &atomicity, retries, recovery, observers, evidence,\\ &coverage, exceptions, bypasses, liveness, remedy,\\ &remainder, revision) \end{aligned} \quad .$$

Coverage is claim-relative. The evaluator must enumerate or otherwise bound the consequential path family, including adapters, caches, retries, fallbacks, concurrency, administrative channels, updates, recovery tools, side channels, direct devices, and external actors. Each excluded path becomes an explicit assumption or remainder. Complete mediation is established only for the bounded family and environment challenged by the evidence.

Semantic adequacy, executor conformance, custody fidelity, and effect integrity remain different courts with different subjects and oracles. A verdict does not issue execution authority. A receipt does not prove the external effect. Atomic

commit inside the governed substrate cannot erase irreversible, delayed, partial, or independently caused external effects; reserve, compensate, reconcile, and classified-remainder outcomes must be first-class. Absence is evidence only with known observation coverage, freshness, liveness, and denominator.

The universal GSM can type
`MEDIATES_CONSEQUENTIAL_PATH`, `BOUNDS_ENFORCEMENT_COVERAGE`, `EXCLUDES_PATH_AS_ASSUMPTION`, `ATTACKS_ENFORCEMENT_BYPASS`, `CLASSIFIES_PARTIAL_EFFECT_REMAINDER`, `OBSERVES_ABSENCE_WITH_DENOMINATOR`, `PROVIDES_CONTEST_AND_REMEDY`, and `INVALIDATES_ENFORCEMENT_CLAIM`.

This reverse-dependency review establishes four editorial commitments: adopt the path-coverage enforcement case; require explicit environment, exclusions, bypass tests, liveness, and remedy; preserve the four court subjects and their authority boundaries; and treat external partial or irreversible effects as classified remainder rather than atomicity success.

The Road Can Encode the Wrong Law

Public orientation

The Mechanics of Constitutional Enforcement argues that consequential rules become stronger when represented in identity-bearing topology, capability release, ratchets, courts, and receipts rather than left as optional predicates. Structural placement can make violations harder. It cannot make the represented constitution correct, complete, legitimate, live, or immune to bypass.

Adversarial tests

1. **Wrong-road attack.** Topology faithfully enforces a mistaken rule. Structuralization must preserve authority, rationale, challenge, amendment, and supersession.
2. **Semantic-homomorphism attack.** A rich ordered obligation is replaced by a counter, flag, or aggregate that accepts invalid histories. The court must test preserved structure, not only final value.
3. **Nonconforming-executor attack.** The executor ignores or corrupts internal law. Consequential effects require mediation outside the executor's self-report.
4. **Coverage-gap attack.** One effect path bypasses the ratchet or court. Complete mediation is always effect-class and boundary relative, with an explicit unmeasured remainder.
5. **Stale-witness attack.** Evidence is valid for an earlier generation, policy, environment, or subject. Freshness and exact binding are part of admissibility.
6. **Absence-fallacy attack.** No event was observed, so non-occurrence is inferred without coverage, liveness, clock, and loss guarantees. Absence becomes evidence only under a declared observation contract.

7. **Atomicity-remainder attack.** An external effect partially occurs before refusal or rollback. Effects need prepare, commit, observation, compensation, and classified remainder.
8. **State-explosion and liveness attack.** Stronger topology makes useful progress computationally or institutionally impossible. Cost and continuation are constitutional properties, not afterthoughts.

Claim and evidence ceiling

The paper establishes a design thesis and a research program. The Ratchet Governance Conjecture remains a conjecture until its system class, semantics, threat model, and proof obligations are fixed. Small examples show failure modes and candidate mechanisms; they do not establish general security, completeness, minimality, or performance.

Claims about enforcement effectiveness, formal verification, attestation, side-channel resistance, liveness, or comparative cost require exact realizations and reproducible evidence. Internal corpus alignment establishes architectural composition only. No external research was added in this pass.

Reader-facing consequence

The practical lesson is not merely that rules should be checked. Where possible, the path to an effect should carry the rule, identity, and evidence so that violating the rule requires leaving a visible governed road. The remaining paths must be named rather than assumed away.

23. Publication Standing and Evidence Limits

Editorial Provenance and Standing

Source, Evidence, and Standing

This is an AI-prepared successor revision candidate derived from one exact author source. The source remains unchanged. The integration custody manifest is
`work/paper-saturation-v1/evidence/mechanics-of-constitutional-enforcement/integration-custody-manifest.v0.1.json`,
SHA-256
`9FF205175F2D2BB69EEAEDC7C2D03ED1775A2D28058BEE84F9599FD09E817CD0`.
A custody manifest binds the source and six exact internal comparison works. Exact hashes establish which texts were reviewed, not that their claims are correct, implemented, independently reproduced, secure, or current. The paper reports a bounded repair involving one specification family, eleven constitutions, seventy-seven operations, and linearizable workflows. No separate recoverable fixture, test corpus, receipt set, or repair record supporting those counts was

identified in the reviewed paper corpus. The counts are therefore retained as **source-reported internal case evidence**, not independently replayed evidence. Copies found inside excluded AOK/AOK-G material were not used as Ableman semantic evidence. The paper is an architectural argument and research program. Its strongest demonstrated claim in this pass is the representational distinction illustrated by the seven-stage example. Claims about hostile executors, independent brokers, plural witnesses, atomic effects, and broad governance expressibility remain design propositions unless tied to an exact implementation and named proof surface.

24. Conclusion: The Constitution as the Road

The motivating repair was small in code and large in meaning.

A counter was replaced by a state chain. No new primitive was added. The kernel remained unchanged. Yet the system moved from counting seven events to representing seven ordered, unique stages.

The lesson is not that counters are defective or that automata are magical. It is that the original representation preserved the wrong invariant.

Where a meaning lives determines how it can fail.

When a governance rule lives in runtime data, its force depends on every consequential path interpreting that data correctly. When the rule lives in topology, a conforming executor has no valid path through which to violate it. When the topology is canonically hash-bound, weakening the rule changes the identity of the constitution. When effects are mediated by independent authority, a nonconforming executor cannot silently transform arbitrary behavior into an authorized consequence. When witnesses and monitors exist outside the executor, missing or contradictory evidence can trigger refusal, rollback, quarantine, or alert.

An absolute residue remains. If every authority, witness, verifier, key, clock, effect boundary, and physical substrate relevant to an event is compromised together, fabricated history may be indistinguishable from faithful execution. No semantic vocabulary abolishes that limit.

But the practical design space is vast before that boundary is reached.

Authority can be divided. Capabilities can be narrowed. Execution can be isolated. Effects can be staged. Commit can be separated from computation. Receipts can be independently signed. Silence can become evidence. Constitutions can be content-addressed. Mutations can be forced to declare themselves as changes of identity.

The resulting aspiration is not a system in which failure becomes metaphysically impossible. It is a system in which failure cannot easily become invisible, unlimited, self-authorizing, or historically legitimate.

A constitution written only as data asks the executor to remember the law.

A constitution written as topology reshapes the space in which execution occurs.

A constitution whose topology is identity-bearing makes semantic weakening visible as constitutional change.

And a constitution whose effects are independently mediated does not merely place a sign beside the forbidden road.

It closes the road, takes away the bulldozer, and stations a witness where new tracks would have to appear.

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *The Three Hash Rule.* Zenodo, 2026. doi:10.5281/zenodo.20788617

Adam Ableman *Constitutional Storage.* Zenodo, 2026. doi:10.5281/zenodo.20788662

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Adam Ableman *Semantic Memory Hierarchies.* Zenodo, 2026. doi:10.5281/zenodo.20793071

Corpus-Wide Reference Positioning

This paper argues that governed systems frequently represent policy as runtime data evaluated by predicates: counters, flags, permission checks, validation functions, and conditional branches. Its nearest external comparison fields are security, governance, provenance. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counter-pressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Jerome H. Saltzer and Michael D. Schroeder. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63(9), 1975, pp. 1278-1308. DOI: 10.1109/PROC.1975.9939. https://doi.org/10.1109/PROC.1975.9939 **Relationship:** Foundational protection principles used as a comparison point for least authority, mediation, and inspectability.

[**E2**] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD dissertation, Johns Hopkins University, 2006. http://www.erights.org/talks/thesis/ **Relationship:** Predecessor object-capability treatment of authority and compositional security.

[**E3**] Gerwin Klein et al. "seL4: Formal Verification of an OS Kernel." *SOSP '09*, 2009, pp. 207-220. DOI: 10.1145/1629575.1629596. https://doi.org/10.1145/1629575.1629596 **Relationship:** Predecessor for machine-checked implementation correspondence and capability-based kernel design.

[**E4**] Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Peter G. Neumann. *An Introduction to CHERI*. University of Cambridge Technical Report UCAM-CL-TR-941, 2019. DOI: 10.48456/tr-941. cl.cam.ac.uk source **Relationship:** Adjacent capability architecture demonstrating unforgeable hardware authority and compartmentalization.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763
Relationship: Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/
Relationship: Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0.* Basic Books, 2006. https://codev2.cc/
Relationship: Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *Effect Sovereignty.* Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Ableman Corpus current edition; work identity `PAPER.PUB.18601631`;
source SHA-256
`6DE46F821594AB11BFB7F8373368C452BD89E5BD4EB172B88D9467E87CF74C33`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Integrity Enforcement Layer.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-505307E9E6571B63`;
source SHA-256
`F858AB32E0937B796353CAE573DD4183FDF4F179A3E1E7E40C4E0C6BF1AB0018`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Constitutional Syscalls.* Ableman Corpus current edition; work identity `PAPER.PUB.18737229`;
source SHA-256
`5F14356180812262778F8574C292F4F6B560993042E5358B02EAB9CF0F7DE14C`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Universal Governance Kernel (UGK).* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-1F9BFB2BF8370226;
source SHA-256
AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Ableman Constitutional Governance.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-79353AF163A4B0F1;
source SHA-256
FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-D2D97C4042FD281F;
source SHA-256
FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Ableman Corpus references [**1**] Ableman Adam Ableman *The Integrity Enforcement Layer.* Exact reviewed author source,
SHA-256
FF7A53289D84BAEC82B89E55A9DA6088A8FD194399ABB6A57BFB739F3008B46B.

[**2**] Ableman Adam Ableman *The Three Hash Rule.* Exact reviewed published author source,
SHA-256
03F60317CC9AFB36DDBCF9BF1C6D185611666CC4D9AB290BD4BCC17DF4579B7C.

[**3**] Ableman Adam Ableman *Reasoning by Way of Receipts.* Exact reviewed author source,
SHA-256
8D884B96597EA6303A324173A7CE2D8C735A952E91FBE4C9EB6509FA6D6B195F.

[**4**] Ableman Adam Ableman *A Formal Theory of Work Custody.* Exact reviewed author source,
SHA-256
49C38F60B95E107F8B3D864ACAFB4C49067A146C807E5248307DE7B009AD4696.

[**5**] Ableman Adam Ableman *Admissibility Engineering.* Exact reviewed author source,
SHA-256
FA80D5895A569839958193735B754BE62EE20E1B399A06CB3BA959614DEBF7A3.

Editorial source notes retained from predecessor

[**6**] Ableman Adam Ableman *Constitutional Storage.* Exact reviewed published author source,
SHA-256
`BEE653947A4C17B3A7F02C4E2EB7F8C03A5E45A18FF3D86E6DB9766555A7FB1C`.

Reference-status note. These internal corpus sources support conceptual comparison and custody. No external research, implementation replay, security audit, independent replication, or product deployment assessment was performed.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848448